SMART EXAMINATION MANAGEMENT SYSTEM

QR-Based Student Verification and Automated Anti-Cheating Seating Allocation

Project Documentation — Simple and Understandable Version

Academic Programme: Bachelor of Technology (B.Tech.) in Computer Science and Engineering
Institute: Government College of Engineering, Amravati (GCOEA)
Project Guide: Dr. Naresh G. Gadge

Project Team

1. Harshali Villhekar
2. Vaishnavi Jadhav
3. Tanvi Raut

---

1. Introduction

Examinations are an important part of the education system. In many colleges, examination management involves several manual activities, such as preparing student lists, allocating seats, generating hall tickets, checking student identity, and recording attendance.

These activities take time and may lead to mistakes. Manual seating arrangements can also place students from the same branch or subject next to one another, depending on the examination requirements.

Our **Smart Examination Management System** is designed to make examination management easier, faster, and more reliable by using web technology, QR codes, and automated seating allocation.

The system provides separate dashboards for administrators, students, and invigilators. Administrators manage examination information, students access their hall tickets, and invigilators use a dedicated Android application to scan QR codes and verify students.

The project uses the MERN stack for the web application and backend. The invigilator Android application will use React Native and Expo and will connect to the same backend and MongoDB database.

1.1 Main Objectives

- Automate examination seating allocation.
- Generate digital hall tickets containing QR codes.
- Verify students by scanning their hall-ticket QR codes.
- Display student photographs and examination details for identity checking.
- Record attendance digitally.
- Reduce duplicate attendance and invalid hall-ticket use.

- Provide an Android application specifically for invigilators.
- Support offline attendance recording when the app and required examination data have been prepared in advance.
- Generate attendance reports for administrators.

2. Abstract

The Smart Examination Management System is a web-based application with a dedicated Android application for invigilators. It is developed to simplify examination activities and reduce the workload of examination staff.

The administrator can manage student records, academic branches, subjects, examination halls, examination schedules, and seating arrangements. Student records can be imported from Excel or CSV files. The system can allocate seats using a branch-mixing strategy to help reduce the possibility of students from the same branch sitting next to each other.

The system generates digital hall tickets containing student information and subject-specific QR codes. Students can access their examination details and download their hall tickets through the student portal.

The invigilator uses a separate Android application to scan a student's QR code. The app sends the scanned information to the Node.js and Express backend, which validates the QR code and checks the student's examination, hall, and seat allocation. The verified information is then displayed on the mobile screen, allowing the invigilator to compare the registered photograph with the student's identity document and record attendance.

The application uses React for the web interface, React Native with Expo for the Android app, Node.js and Express for backend operations, and MongoDB for data storage. Offline attendance storage and synchronization can also be implemented so that records captured without network connectivity can be uploaded when the connection becomes available again.

The main goal is to improve the speed, accuracy, security, and convenience of examination management.

3. Problem Statement

Traditional examination management systems have several problems.

3.1 Manual seating allocation

Examination staff may need to prepare seating arrangements manually for hundreds of students. This process takes time and may result in duplicate or incorrect seat assignments.

3.2 Fake hall tickets

Paper hall tickets can be copied, modified, or presented by an unauthorized person. A verification system is needed to check whether a hall ticket belongs to a registered student and a particular examination.

3.3 Manual attendance

Invigilators often use paper attendance sheets. Checking each student and updating the attendance register can be time-consuming.

3.4 Duplicate attendance

Without proper verification, an attendance record may accidentally be entered more than once.

3.5 Network problems

If the internet connection fails during an examination, an online-only application may stop working. The system therefore needs a way to store prepared examination data locally and save attendance temporarily when offline.

4. Proposed System

Our proposed system combines automated examination management with QR-based student verification.

It consists of four main parts:

1. **Admin web dashboard:** Manages students, exams, halls, seating allocation, staff, and reports.
2. **Student web dashboard:** Allows students to view exam schedules, allocated seats, and hall tickets.
3. **Invigilator Android app:** Scans QR codes, displays student information, and supports attendance recording.
4. **Backend and database:** Processes requests, verifies records, applies access permissions, and stores official information.

All three user interfaces use the same backend and database. This avoids maintaining separate student and attendance records for the mobile app.

5. Technology Stack

Technology	Purpose
React	Admin and student web dashboards
Tailwind CSS	Web interface styling
React Native	Invigilator Android application
Expo	Mobile app development and Android builds
Expo Camera	Mobile camera and QR scanning
Node.js	Backend JavaScript runtime
Express.js	REST API and business logic
MongoDB	Database for students, exams, seats, and attendance
Mongoose	MongoDB data models and validation
JWT	Authentication tokens
bcrypt	Password hashing
xlsx	Reading Excel and CSV student data

QR-code library	Generating hall-ticket QR codes
jsPDF	Generating downloadable hall tickets
Local mobile storage	Saving prepared examination data and pending attendance offline

Important: These are the planned or intended technologies. The final report should list a feature as implemented only after it has been completed and tested.

6. System Users and Their Responsibilities

6.1 Administrator

The administrator manages the complete examination process.

Main functions:

- Log in securely.
- Add, update, and manage student records.
- Import student records from Excel or CSV files.
- Manage academic branches and subjects.
- Create examination halls and configure their seating capacity.
- Schedule examinations.
- Generate seating allocations.
- Generate hall tickets with QR codes.
- Create invigilator accounts and assign examinations and halls.
- View attendance records.
- Generate examination-wise and hall-wise reports.

6.2 Student

The student uses the student portal to access personal examination information.

Main functions:

- Log in to the student dashboard.
- View personal details.
- Check the examination timetable.
- View the allocated examination hall and seat number.
- Download the digital hall ticket.
- Present the hall-ticket QR code for verification.

Students should only be able to access their own permitted information.

6.3 Invigilator

The invigilator uses the dedicated Android app during examinations.

Main functions:

- Log in using authorized invigilator credentials.
- View assigned examinations and halls.
- Open the QR scanner directly after login.
- Scan student hall-ticket QR codes.
- View the student's name, PRN, photograph, exam, hall, and seat.
- Compare the registered photograph with the student's identity document.
- Record attendance after checking the student's identity.
- Receive warnings for invalid QR codes, wrong halls, and duplicate attendance.
- Store attendance locally when offline support is available.
- Synchronize pending attendance records when connectivity returns.

The app must restrict each invigilator to authorized examination duties.

7. System Architecture

The system has three main technical layers.

7.1 Frontend Layer

This is the part users see and interact with.

It includes:

- React web dashboards for administrators and students.
- A React Native Android application for invigilators.
- Forms, tables, hall-ticket displays, QR scanning screens, and attendance results.

7.2 Backend Layer

The Node.js and Express backend receives requests from the web application and Android app.

It performs operations such as:

- Authenticating users.
- Checking user roles and permissions.
- Validating QR codes.
- Retrieving student and examination details.
- Creating seating allocations.
- Recording attendance.
- Processing offline synchronization requests.
- Generating reports.

7.3 Database Layer

MongoDB stores the official records, including:

- Users and roles.
- Students and branches.
- Subjects and examinations.
- Examination halls and seats.
- Student seat allocations.
- Attendance records.

Overall data flow

1. The administrator adds students and schedules an examination.
2. The backend stores the information in MongoDB.
3. The seating algorithm allocates students to available seats.
4. Hall tickets are generated with QR codes linked to examination records.
5. The invigilator scans a QR code using the Android app.
6. The backend verifies the QR code and retrieves the official details.
7. The app displays the result.
8. Attendance is saved to MongoDB when online, or stored locally for later synchronization when offline support is implemented.

8. Automated Seating Allocation

8.1 What is seating allocation?

Seating allocation is the process of assigning each eligible student to a particular examination hall and seat.

For example, suppose an examination has 12 students from three branches:

- Computer Science: 4 students
- Electronics: 4 students
- Mechanical: 4 students

The system groups students by branch, sorts them using their PRN or roll number, and interleaves the groups.

An illustrative ordering could be:

Seat order	Student branch
Seat 1	Computer Science
Seat 2	Electronics
Seat 3	Mechanical

Seat 4	Computer Science
Seat 5	Electronics
Seat 6	Mechanical

The pattern continues while eligible students remain.

8.2 How the algorithm works

1. Retrieve eligible students for the examination.
2. Group students by academic branch.
3. Sort each branch group by PRN or roll number.
4. Select students from the groups in round-robin order.
5. Retrieve the available seats from the selected halls.
6. Assign students to seats.
7. Check that no student has been allocated twice for the same exam.
8. Check that no seat has been assigned to multiple students in the same exam.
9. Save the allocations in MongoDB.

If there are more students than available seats, the system should stop and report insufficient seating capacity.

8.3 How seating helps reduce cheating

Mixing students from different branches can reduce the chance that students from the same branch sit next to one another.

However, round-robin grouping alone does not guarantee that every adjacent student has a different branch or subject, particularly when group sizes are uneven or seats wrap to a new row. A final seat-by-seat validation step is needed if the college requires strict adjacency constraints.

Different branches also do not automatically mean different examination papers. The algorithm should use the actual examination and subject information when enforcing those rules.

9. Hall Ticket and QR Code Generation

9.1 Hall ticket information

A hall ticket may contain:

- College name and logo.
- Student name.
- PRN and roll number.
- Branch and semester.
- Student photograph.
- Examination schedule.

- Subject names and codes.
- Assigned hall and seat number.
- QR code for verification.

The hall ticket can be generated as a downloadable PDF.

9.2 How the QR code works

A QR code contains information that the scanner can decode. In this project, it should contain a secure token or a reference linked to the student's particular examination allocation.

The QR code should not be trusted merely because it contains a student's PRN or other readable details.

A secure verification process is:

1. The system generates a unique token for the examination allocation.
2. The backend stores the token or its secure hash against the relevant allocation.
3. The QR code is printed on the hall ticket.
4. The invigilator scans the code using the Android app.
5. The app sends the scanned token to the backend.
6. The backend checks that the token is valid and that the allocation belongs to the active examination.
7. The backend verifies the assigned hall and seat.
8. The app displays the verified student information.

The QR code should be tied to the appropriate exam so that a code issued for one exam cannot automatically be used for another.

9.3 What happens when the QR code is scanned?

A successful verification may display:

- Student name.
- PRN.
- Registered photograph.
- Examination name.
- Examination hall.
- Assigned seat.
- Attendance status.

If the QR code is invalid, revoked, or associated with another examination or hall, the app should display a warning and block attendance marking.

10. Invigilator Android Application

The invigilator app is a separate Android application built with React Native and Expo.

It connects to the same Express API and MongoDB database as the existing website.

10.1 Main screens

Login screen

The invigilator enters authorized credentials. The backend checks the account and returns an authentication token.

Scanner screen

After login, the app opens the QR scanner. Camera permission is requested from the operating system when required.

Student verification screen

After scanning, the app displays the student's verified details and registered photograph.

Attendance screen

The invigilator confirms the student's identity and records attendance. The backend checks for duplicate records.

Offline and synchronization screen

If offline attendance support is implemented, the app shows whether the exam package is available locally, how many records are pending synchronization, and whether synchronization succeeded.

10.2 How the app connects to the existing project

The connection is:

Android app → Express API → MongoDB

The app does not connect directly to MongoDB.

For example, the existing backend may expose the following routes:

- POST /api/auth/login
- POST /api/attendance/verify-qr
- POST /api/attendance/mark
- GET /api/attendance/hall-roster/:examId/:hallId
- POST /api/attendance/sync-offline

These routes are described in the proposed design. Before implementation, they must be checked against the actual backend code.

11. Offline Attendance and Network Failure

11.1 Why offline support is needed

If the phone loses its network connection during an examination, it may not be able to contact the backend.

The app therefore needs a way to work with examination information downloaded before entering the hall.

11.2 Pre-examination preparation

Before the examination:

1. The invigilator logs in while connected to the authorized network.
2. The app downloads the assigned examination and hall roster.
3. It stores the required student details, seat allocations, photographs where permitted, and examination information locally.
4. The app checks that the data has downloaded successfully.
5. The invigilator confirms that the device is ready for offline operation.

11.3 During the examination

If connectivity is lost:

1. The scanner reads the QR code locally.
2. The app compares the QR information against the previously downloaded roster.
3. The app displays the locally available student and seat information.
4. The invigilator checks the student's physical identity.
5. The attendance event is stored in the device's local database.
6. The app indicates that the attendance record is pending synchronization.

Offline verification is based on previously downloaded data. It cannot guarantee a fresh central-server check while the server is unreachable.

11.4 After connectivity returns

1. The app identifies pending attendance records.
2. It sends them to the Express backend.
3. The backend checks the examination, hall, student, and invigilator permissions.
4. It detects duplicate or conflicting records.
5. Valid records are saved in MongoDB.
6. The backend returns a synchronization summary.
7. The app marks successfully synchronized records as complete.

11.5 Technology for offline storage

Because the invigilator application is an Android app, it can use a local mobile database such as SQLite through a compatible React Native library.

The original PWA design uses browser IndexedDB. IndexedDB is appropriate for the web version, but it is not the native storage mechanism for the Android app.

Offline synchronization requires additional code; installing Expo or a QR scanner library does not automatically provide offline attendance.

11.6 Important network limitation

A signal jammer may block cellular data and Wi-Fi. If the invigilator's phone cannot reach the server, offline mode may still allow local scanning of cached information.

A campus server connected over wired Ethernet can provide a local service, but wireless devices can connect to it only if a usable wireless network remains available. The network design must be tested in the actual examination environment.

12. Database Design

The main MongoDB collections are described below.

Collection	Purpose
User	Login credentials, roles, and staff assignments
Student	Student details, PRN, branch, and photograph
Branch	Academic branches such as CSE and EXTC
Subject	Subject name, code, branch, and semester
Exam	Examination date, subject, semester, and timings
Hall	Hall name, building, and seating capacity
Seat	Seat number and row/column coordinates
Allocation	Student, exam, hall, and seat assignment
Attendance	Attendance status, verification time, and audit details

Important database rules

- Each student should have a unique PRN.
- Each examination should have at most one allocation per student.
- A seat should not be assigned to two students in the same examination.
- Attendance should have a unique constraint for the intended exam and student combination.
- Attendance should record the authorized invigilator and verification method.
- Offline records should include their local recording time and synchronization status.

Database indexes help prevent duplicate records, but the application must also handle duplicate-key errors and conflicts correctly.

13. Main API Endpoints

The backend exposes REST APIs so the web and mobile applications can communicate with it.

Module	Example endpoint	Purpose
Authentication	POST /api/auth/login	Authenticate a user
Authentication	GET /api/auth/me	Retrieve the current user's profile
Students	GET /api/students	Retrieve authorized student records
Halls	GET /api/halls	Retrieve hall information
Subjects	GET /api/subjects	Retrieve subject information
Exams	GET /api/exams	Retrieve examination schedules
Seating	POST /api/allocations/generate	Generate seating allocations
QR verification	POST /api/attendance/verify-qr	Verify a scanned QR code
Attendance	POST /api/attendance/mark	Record attendance
Offline sync	POST /api/attendance/sync-offline	Synchronize locally stored records
Roster	GET /api/attendance/hall-roster/:examId/:hallId	Retrieve an assigned hall roster
Reports	GET /api/attendance/report/:examId	Retrieve an authorized attendance report

These endpoints form the intended API design. Your team must verify the actual route names, request fields, and response structures before using them in the Android application.

14. Security Requirements

The system handles student records and official attendance, so security is important.

- Store passwords as secure hashes using bcrypt or a suitable password-hashing method.
- Use authentication tokens and verify them on protected API routes.
- Enforce role-based authorization on the backend.
- Restrict invigilators to their assigned examinations and halls.
- Do not trust student IDs or examination details supplied only by the mobile client.
- Use secure, unpredictable QR tokens or cryptographically signed payloads.
- Prevent duplicate attendance using database constraints and safe server-side operations.
- Use HTTPS for production API communication.
- Avoid putting database credentials, JWT secrets, or private API keys in mobile application code.
- Protect cached student photographs and examination data on devices.
- Keep an audit trail of attendance changes and offline synchronization.
- Do not publish real passwords in project documentation or demonstration screenshots.

A QR code containing ordinary JSON fields is not cryptographically secure by itself. The backend must validate the token or signature.

15. Testing and Validation

The system should be tested before being used in a real examination.

Test case	Expected result
Valid QR for an assigned student	Display the correct verified student information
Invalid or unknown QR	Display an error and block attendance
QR from another examination	Display an examination mismatch warning
Student assigned to another hall	Display a hall mismatch warning
Student already marked present	Prevent duplicate attendance
Unauthorized invigilator	Deny access to protected records
Missing camera permission	Explain how to enable camera access
Network lost after roster download	Use local data if offline functionality is implemented
Network lost before roster download	Warn that the device is not ready for offline operation
Network restored	Synchronize pending records and report conflicts
Insufficient seats	Stop allocation and report the capacity problem
Duplicate seat allocation attempt	Reject the conflicting allocation

The final report should include screenshots and test results from the actual running application.

16. Project Implementation Plan

The project should be implemented in stages.

Phase 1: Inspect the existing project

- Review the React frontend.
- Review the Node.js and Express backend.
- Check authentication and role permissions.
- Identify the MongoDB models.
- Check the existing QR-generation and attendance APIs.

Phase 2: Complete the backend

- Confirm QR verification logic.
- Confirm seating allocation rules.
- Add or correct attendance APIs.
- Enforce duplicate prevention and invigilator permissions.
- Test the APIs independently.

Phase 3: Build the Android application

- Create a separate React Native + Expo project.
- Implement invigilator login.
- Add camera permission handling.
- Add the QR scanner.

- Connect the scanner to the existing backend.
- Display verified student details.

Phase 4: Implement attendance

- Add the attendance confirmation screen.
- Save attendance through the backend.
- Handle duplicate scans and invalid QR codes.
- Show clear success and error messages.

Phase 5: Implement offline support if required

- Download assigned examination data before the exam.
- Store the roster and pending attendance locally.
- Validate scans against cached data while offline.
- Synchronize records after connectivity returns.
- Test conflicting and duplicate submissions.

Phase 6: Testing and APK generation

- Test on a real Android phone.
- Verify the API connection and camera behavior.
- Test security and error cases.
- Generate the APK using Expo EAS Build.
- Install and test the APK on the invigilator's phone.

17. Expected Outcomes

When the required features have been implemented and tested, the system should:

- Reduce manual examination administration.
- Provide organized seating assignments.
- Make hall-ticket verification faster.
- Help invigilators identify incorrect examination or hall assignments.
- Reduce duplicate attendance records.
- Give students convenient access to hall tickets and examination details.
- Provide administrators with attendance reports.
- Support offline attendance capture and later synchronization, subject to the implemented offline design.

These outcomes depend on successful implementation and testing. The project should not claim complete offline operation, guaranteed prevention of cheating, or zero downtime without supporting test evidence.

18. Conclusion

The Smart Examination Management System combines a web application with an Android application to improve the management of college examinations.

The administrator manages student records, examination schedules, halls, seating allocations, and reports. Students use the portal to access examination details and hall tickets. Invigilators use the Android application to scan QR codes, verify student information, and record attendance.

The existing React frontend, Node.js and Express backend, and MongoDB database form the foundation of the system. The new React Native + Expo app connects to the same backend, allowing attendance recorded from the phone to appear in the administrator's dashboard.

The project can also support offline attendance through local storage and synchronization when connectivity returns. This functionality must be implemented and tested carefully.

The overall aim is to make examination administration more efficient, verification more reliable, and attendance records easier to maintain.

---

19. Simple Viva Explanation

Question: What is your project?

Our project is a Smart Examination Management System. It automates student management, examination scheduling, seating allocation, hall-ticket generation, QR verification, and attendance recording.

Question: Why do you need a QR code?

The QR code links a student's hall ticket to the relevant examination record. When an invigilator scans it, the backend checks the student's details, examination, hall, and seat allocation.

Question: Why are you creating a separate Android app?

We are creating a dedicated Android app so that invigilators can open the scanner directly on their phones, scan hall-ticket QR codes, view verified student details, and record attendance.

Question: How does the Android app connect to your website?

The app sends requests to the same Node.js and Express backend used by our React website. The backend retrieves or updates the official records in MongoDB.

Question: What is automated seating allocation?

It is the process of assigning eligible students to available seats using a programmed algorithm. Our planned branch-mixing strategy attempts to separate students from the same branch, subject to the seating constraints.

Question: How will the system work without internet?

Before the examination, the app can download the assigned hall roster. If offline functionality is implemented, it can use that cached data to verify the local roster and store attendance on the phone. When connectivity returns, pending records are synchronized with the backend.

Question: Does the QR code alone prove a student's identity?

No. The backend verifies the QR token and examination allocation. The invigilator must also compare the registered photograph with the student's identity document.

Question: What is the main advantage of your project?

It reduces manual work, makes student verification faster, improves attendance management, and helps organize examination seating.

Documentation note: This is a simplified implementation plan based on the project description provided. Before submitting it as a final project report, update the implementation status, actual API endpoints, database fields, screenshots, test results, and offline capabilities to match the code your team has completed.